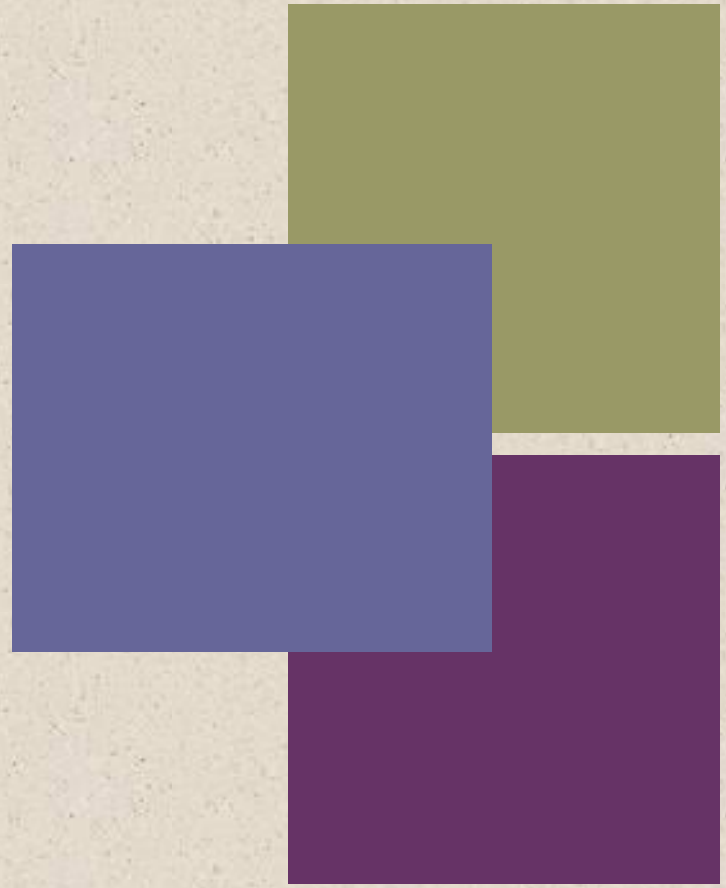


William Stallings Computer Organization and Architecture 10th Edition

My notes in Red (please check for updates

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Memory access takes more than 100 clock cycles (DDR delivers in about 50 clock cycles, DDR 3 and DDR are much faster) while processor speed is 1 clock cycle. In early 1980s the speed of both was about the same. So, accessing main memory today is very expensive.



Chapter 4
Cache Memory

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Memory usage

- CPU □ □ Main Memory

  Data & Address (Remember Registers and Busses used)

  CPU □ □ Cache □ □ Main memory

  Data most used (important are kept in Cache

  A CPU may have several cores

  Each core will have its own L1 cache for instructions and L1 cache for data

  L2 can be shared between the instruction and data cache for each core

  L3 will be shared by all cores.

  Intel I7 has 32 Kbytes of L1 caches, 256 Kbytes L2 caches, 8Mbytes of L3 cache. These vary as years go by. http://web.cs.wpi.edu/~cs4515/d15/Protected/LecturesNotes_D15/Week3_TeamA_i7-Presentation.pdf

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Principles behind how cache works

- **Temporal locality** - re-use of specific data over and over
- **Spacial locality** - use of data within close storage locations
- See a for loop. For (…..) { sum=sum+x[i]}
- When CPU read Cache Hit speeds up operation, Cache Miss slows down operation.
- When CPU writes down 2 ways:
  - Write-through Written to both Cache and memory simultaneously.
  - Write-Back Cache waits and writes only when the cache location is evicted (keeps a dirty bit)

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Location Internal (e.g. processor registers, cache, main memory) External (e.g. optical disks, magnetic disks, tapes) **Capacity** Number of words Number of bytes **Unit of Transfer** Word Block **Access Method** Sequential Direct Random Associative	**Performance** Access time Cycle time Transfer rate **Physical Type** Semiconductor Magnetic Optical Magneto-optical **Physical Characteristics** Volatile/nonvolatile Erasable/nonerasable **Organization** Memory modules

Table 4.1
Key Characteristics of Computer Memory Systems

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Characteristics of Memory Systems

- Location
  - Refers to whether memory is internal and external to the computer
  - Internal memory is often equated with main memory
  - Processor requires its own local memory, in the form of registers. Control unit has its own micro-memory.
  - Cache is another form of internal memory
  - External memory consists of peripheral storage devices that are accessible to the processor via I/O controllers
- Capacity
  - Memory is typically expressed in terms of bytes
- Unit of transfer
  - For internal memory the unit of transfer is equal to the number of electrical lines into and out of the memory module. If transferred either as a word or block, it will have to be further subdivided into bytes by assigning offsets.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Method of Accessing Units of Data

Sequential access

- Memory is organized into units of data called records
- Access must be made in a specific linear sequence
- Access time is variable

Direct access

- Involves a shared read-write mechanism
- Individual blocks or records have a unique address based on physical location
- Access time is variable

Random access

- Each addressable location in memory has a unique, physically wired-in addressing mechanism
- The time to access a given location is independent of the sequence of prior accesses and is constant
- Any location can be selected at random and directly addressed and accessed
- Main memory and some cache systems are random access

Associative

- A word is retrieved based on a portion of its contents rather than its address
- Each location has its own addressing mechanism and retrieval time is constant independent of location or prior access patterns
- Cache memories may employ associative access

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Capacity and Performance:

The two most important characteristics of memory

Three performance parameters are used:

Access time (latency)

- For random-access memory it is the time it takes to perform a read or write operation
- For non-random-access memory it is the time it takes to position the read-write mechanism at the desired location

Memory cycle time

- Access time plus any additional time required before second access can commence
- Additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively
- Concerned with the system bus, not the processor

Transfer rate

- The rate at which data can be transferred into or out of a memory unit
- For random-access memory it is equal to 1/(cycle time)

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Memory



- The most common forms are:
  - Semiconductor memory
  - Magnetic surface memory
  - Optical
  - Magneto-optical
- Several physical characteristics of data storage are important:
  - Volatile memory
    - Information decays naturally or is lost when electrical power is switched off
  - Nonvolatile memory
    - Once recorded, information remains without deterioration until deliberately changed
    - No electrical power is needed to retain information
  - Magnetic-surface memories
    - Are nonvolatile
  - Semiconductor memory
    - May be either volatile or nonvolatile
  - Nonerasable memory
    - Cannot be altered, except by destroying the storage unit
    - Semiconductor memory of this type is known as read-only memory (ROM)
- For random-access memory the organization is a key design issue
  - Organization refers to the physical arrangement of bits to form words

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Memory Hierarchy

- Design constraints on a computer's memory can be summed up by three questions:
  - How much, how fast, how expensive
- There is a trade-off among capacity, access time, and cost
  - Faster access time, greater cost per bit
  - Greater capacity, smaller cost per bit
  - Greater capacity, slower access time
- The way out of the memory dilemma is not to rely on a single memory component or technology, but to employ a memory hierarchy

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Register read/write .5 clock cyscles.

Cache access time 1clock cycle or more

DDR 3200MB/s, DDR2 6400MB/s,
DDR3 14928 MB/s, DDR4 25600 MB/s.
These are peak transfer rates.

A Gig costs $5,000 or more

A Gig of DDR3 around $14.00

About 4cents/GB

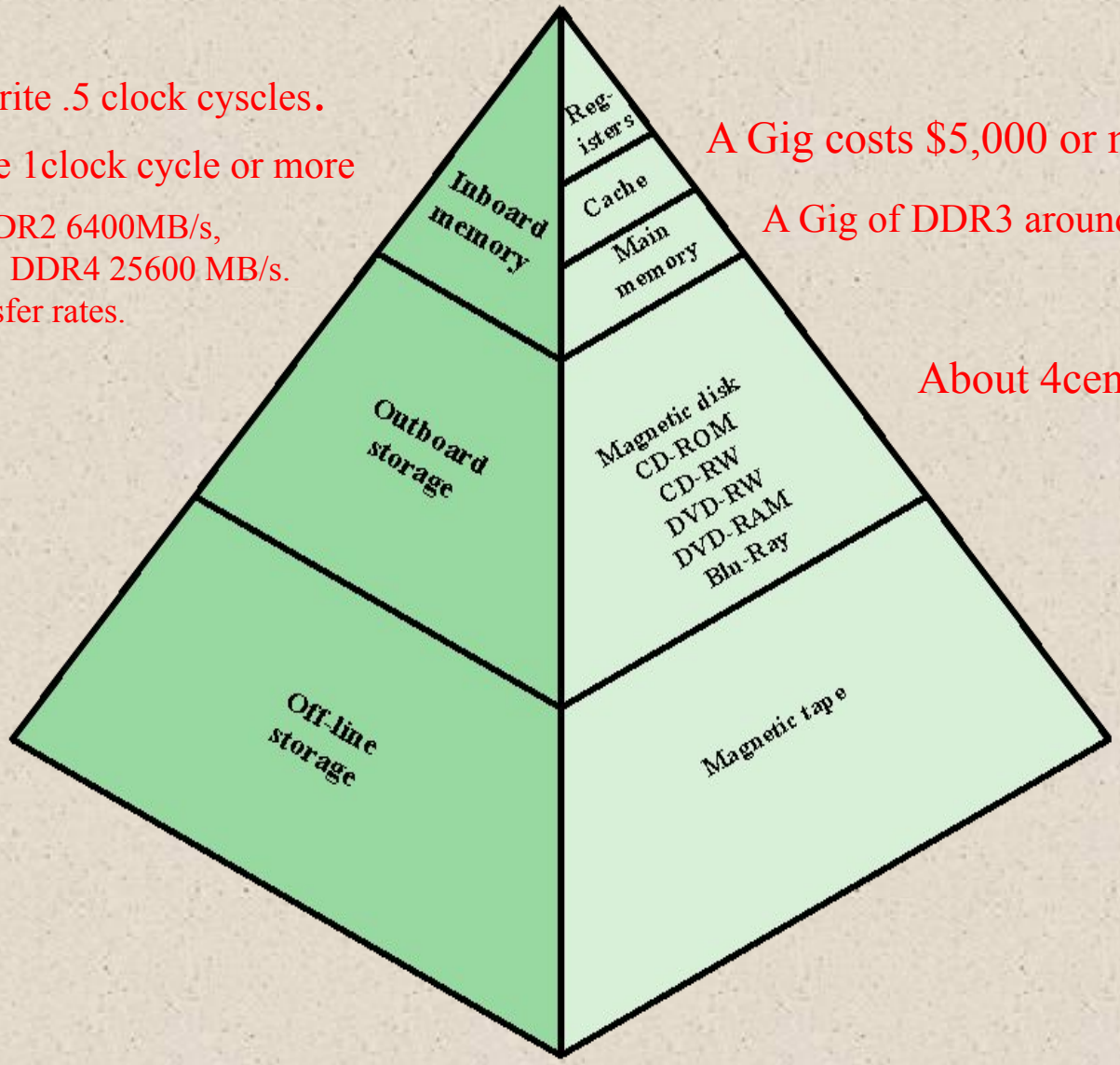


Figure 4.1 The Memory Hierarchy

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Memory

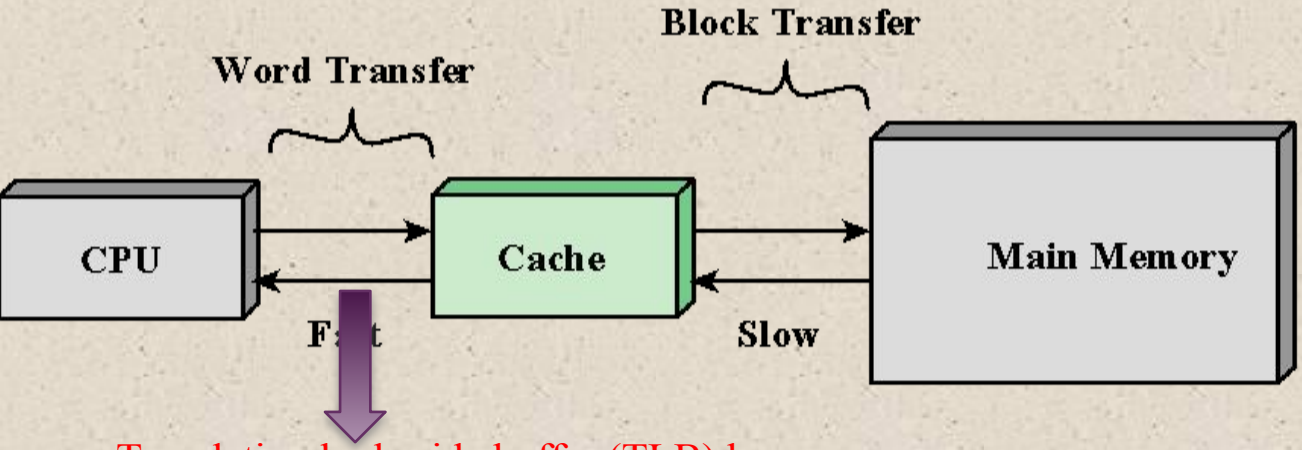
- The use of three levels exploits the fact that semiconductor memory comes in a variety of types which differ in speed and cost
- Data are stored more permanently on external mass storage devices
- External, nonvolatile memory is also referred to as **secondary** memory or **auxiliary** memory
- Disk cache
  - A portion of main memory can be used as a buffer to hold data temporarily that is to be read out to disk
  - A few large transfers of data can be used instead of many small transfers of data
  - Data can be retrieved rapidly from the software cache rather than slowly from the disk

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

DRAM/SRAM

- DRAM must be refreshed every 8 ms or s. Addressed by Row and column. 1 transistor per cell, so cheaper.
- SRAM Fast. 6 transistor per cell, higher power. Does not need to be refreshed. Retains content as long as power is provided.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

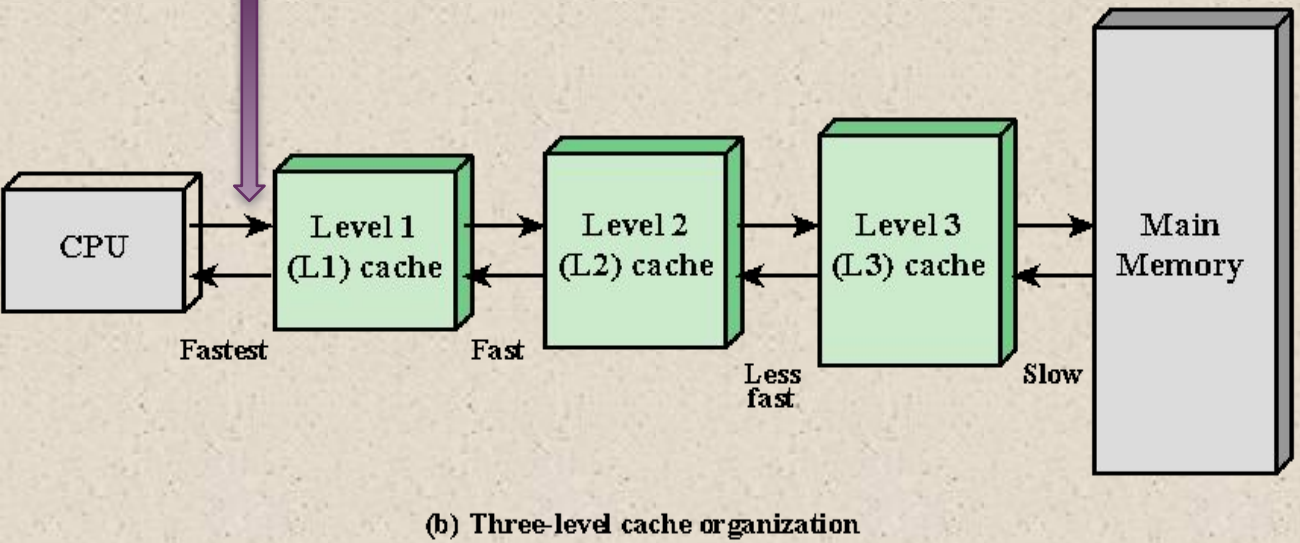


Block/line: A block is multiple words – perhaps 64 bytes or 8 words or more.
Word is 8 byes in 64-bit architecture.

Translation lookaside buffer (TLB) here

(a) Single cache

Refer to:
http://web.cs.wpi.edu/~cs4515/d15/Protected/LecturesNotes_D15/Week3_TeamA_i7-Presentation.pdf



(b) Three-level cache organization

Data is read from closest in hierarchy. Does not go to another layer directly. Data contained one level is also in all other lower levels

Figure 4.3 Cache and Main Memory

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Suppose we have 16 byte Main Memory and 4 byte cache

16 bytes of Main Memory will be numbered 0 to 15, or 1 to F (1111)

we need 2 bits to keep track of the 4 bytes of cache from 0 to 11b

Therefore least 2 significant bits will be used to keep track of cache line

and the two most significant bits will be used keep the tag.

Given any address we will know which cache line will hold the value

and at that location we can inspect the tag to see if the right address is represented

This is the principle behind direct mapping that we will discuss later.

	Cache				Memory	
cache line	valid bit	tag	data		0000	value 0
0		00	value 0		0001	value 1
1		00	value 1		0010	value 2
I will explain rest in class					0011	value 3
0		01	value 4		0100	value 4

Cache Mapping

- Direct mapping – Each block from the memory can go to specific line in the cache. Thus, different blocks will map to the same line and only one block can be placed there at any given time.
- Fully Associative mapping – Any block can go into any cache line
- Set-associative mapping. Certain blocks from memory can go into a set of lines (anywhere within the set).

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Example of Fully Associative Mapping

- Suppose we have 4 GB of memory. To address it we need 32 bits. Check it out 2^{32} = 4,294,967,296 bytes.
- Suppose we have some data in memory location (I separate for readability) 1010 1100 1011 0011 0101 1000 0000 which is given in hex A C B 3 5 8 0 the value inside of it lets call value1. Between the data and address we need add 2 bits, valid bit and dirty bit.
- If it is made into 64 byte blocks, the last (lsb) six bits (000000) will be used to indicate byte offsets, 000 001 010 011 100…111111). The remaining 26 bits are the block address. This is referred to as the Tag (26 bits).
- If we have 32KB of Cache, we will have 512 lines (divide 2^15 by 2^6 which gives us 2^9). All the cache is 1 set in fully associative (not further divided).

Tag	Lines are all together	Byte offset
26 bits	No bits to indicate lines	6 bits

- Comparator compares all 512 lines in parallel to check if any line match the Tag. If so,it gives out the value in that address.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

2_2=4
2_3=8
2_4=16
2_5=32
2_6=64
2_7=128
2_8=256
2_9=512
2_{10}=1024
2_{11}=2048
2_{12}=4096
2_{13}=8192
2_{14}=16384
2_{15}=32768
2_{16}=65536
2_{17}=131072
2_{18}=262144
2_{19}=524288
2^20=1048576
2^24=16777216
2^32 = 4294967296

Direct memory mapping.

To calculate which line will store the block use modulo arithmatic. If there are 1024 blocks and 64 lines of cache take any block, say 963; 963 mode 64 = 3. it will be stored in line 3.

Memory 4Gb 32bit address. We need to make 3 fields:tag, Index, offset.

64 byte block yeilds an offset of 6 bits. Remaing 26 bits will make the tag and index.

Cache is 2^15 and line size of 64 bytes we have 2^9 lines or 512 lines. W need 9 bits to represent lines of cache.

Thus the index is 9

26-9 =17 will be the tag

Tag	Index	Offset
17	9	6

Given an address of 1010 1100 1011 0011 0101 1000 0101 1110

tag	index	offset
1010 00101100110	101100001	011110

$2^2=4$
$2^3=8$
$2^4=16$
$2^5=32$
$2^6=64$
$2^7=128$
$2^8=256$
$2^9=512$
$2^{10}=1024$
$2^{11}=2048$
$2^{12}=4096$
$2^{13}=8192$
$2^{14}=16384$
$2^{15}=32768$
$2^{16}=65536$
$2^{17}=131072$
$2^{18}=262144$
$2^{19}=524288$
2^20=1048576
2^24=16777216
2^32 = 4294967296

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Operation of Cache

- CPU does not know cache exists. It gives the address of data required to be loaded.
- Cache intercepts the request, and the tag, index and offset are extracted from the memory address as in the previous slide.
- Cache uses the index to fine the line where the data from that address 'should' be located. It retrieves that data along with the tag and compares the tag portion. If Tag in the address issued and the tag retrieved from the line are the same then we have a hit. If different we have a miss. If it is a hit use the offset to extract the right byte and send it to the CPU.
- If it is a miss, evict that line and bring in the appropriate block into the line.
- Thrashing: When two addresses are required by CPU alternating, then eviction and reloading constantly happens and it is very expensive. In such case, associative and set-associate is better.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Set Associative Cache. 4-way example

- The number of cache lines are divided into sets. In the previous example we had 512 lines. We will make 128 sets of 4 lines each. Data fetched can go into any of these 4 lines.
- Thus, Tag is 19, Index is 7, and byte offset is 6.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Example problems

- Now let's try some examples (say, Intel I7):
  - Memory Address Size: 64 bits (can access 16 exabytes)
  - Cache Size: 8 MB (8,388,608 bytes), each line of 64 bytes.
  - Number of lines = 8,388,608 (2^23)/64 (2^6) = 131,072 (2^17)
  - Calculate Tag, Index and Offset for Direct Mapping.

Tag	Index	Byte Offset
41	17	6

Same in 8-way associative cache Index will be 14, Tag will be 44
Fully associative will be 58 bits for tag and 6 for offset.

Another Example: RAM 4GB. Cache 64KB, Block/line size: 64 bytes. Find Tag Index and offset for Direct, 4-way set associative, and associative
Solution: Memory blocks 2^32/2^6 = 2^26. Cache lines: 2^16/2^6 = 2^10

Tag	Index	Byte Offset
16	10	6

2-way cache index will become 9 bits and tag will become 17 bits

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

What component manages each memory hierarchy?

- Registers – manipulated by CPU as instructions given by compiler or programmer.
- Cache – Cache controller hardware. It controls what comes in and goes out of the cache, keeps valid bit/dirty bits. Each core has its own cache (L1), L2 cache is shared cache and the controller has to manage coherence problem. If two cores gets value of x, and core 1 updates it, we have a coherence problem. Value stored in cache of the second core should be invalidated.
- Main Memory – controlled by operating system at the request of the operator or a program.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

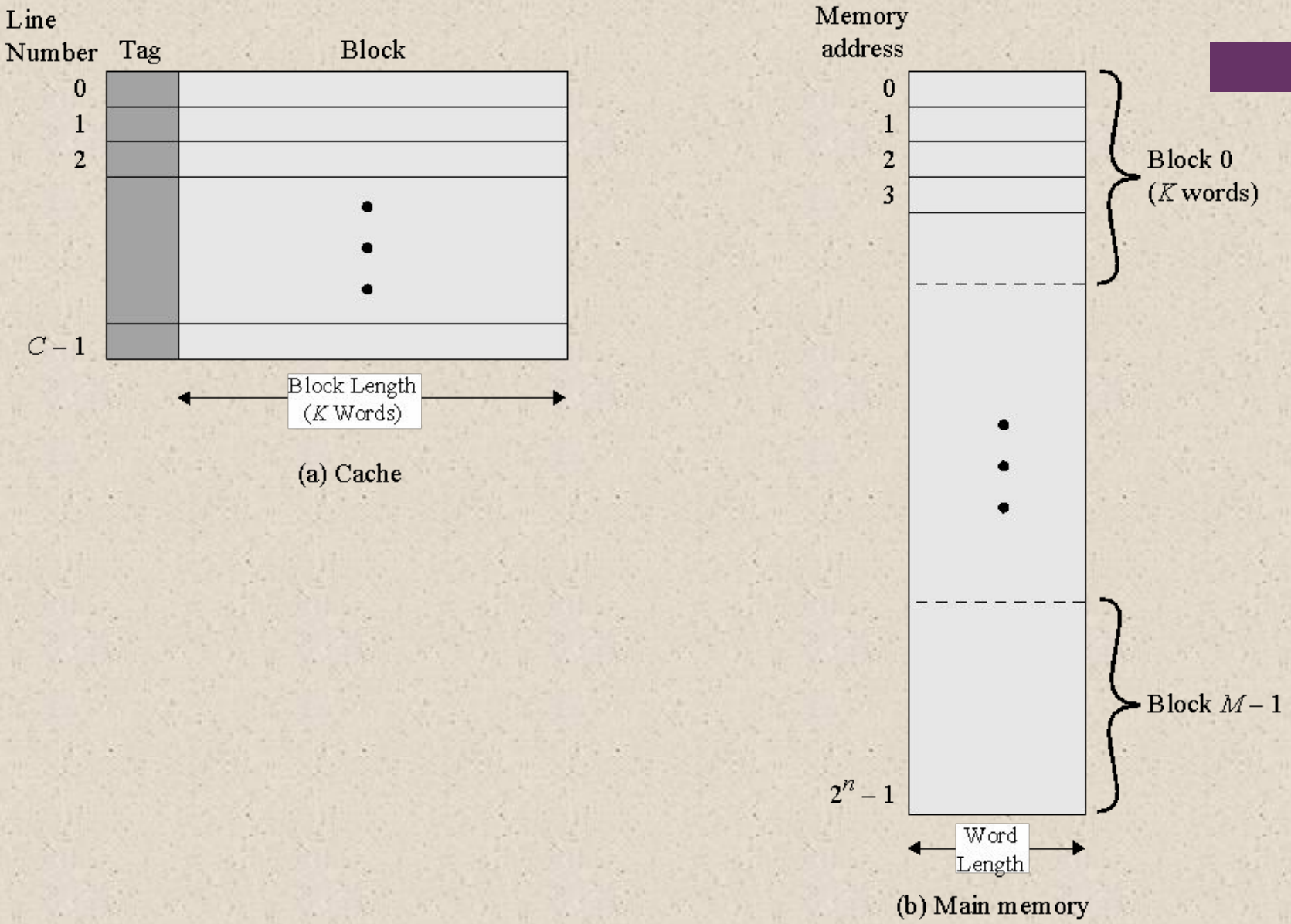


Figure 4.4 Cache/Main-Memory Structure

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

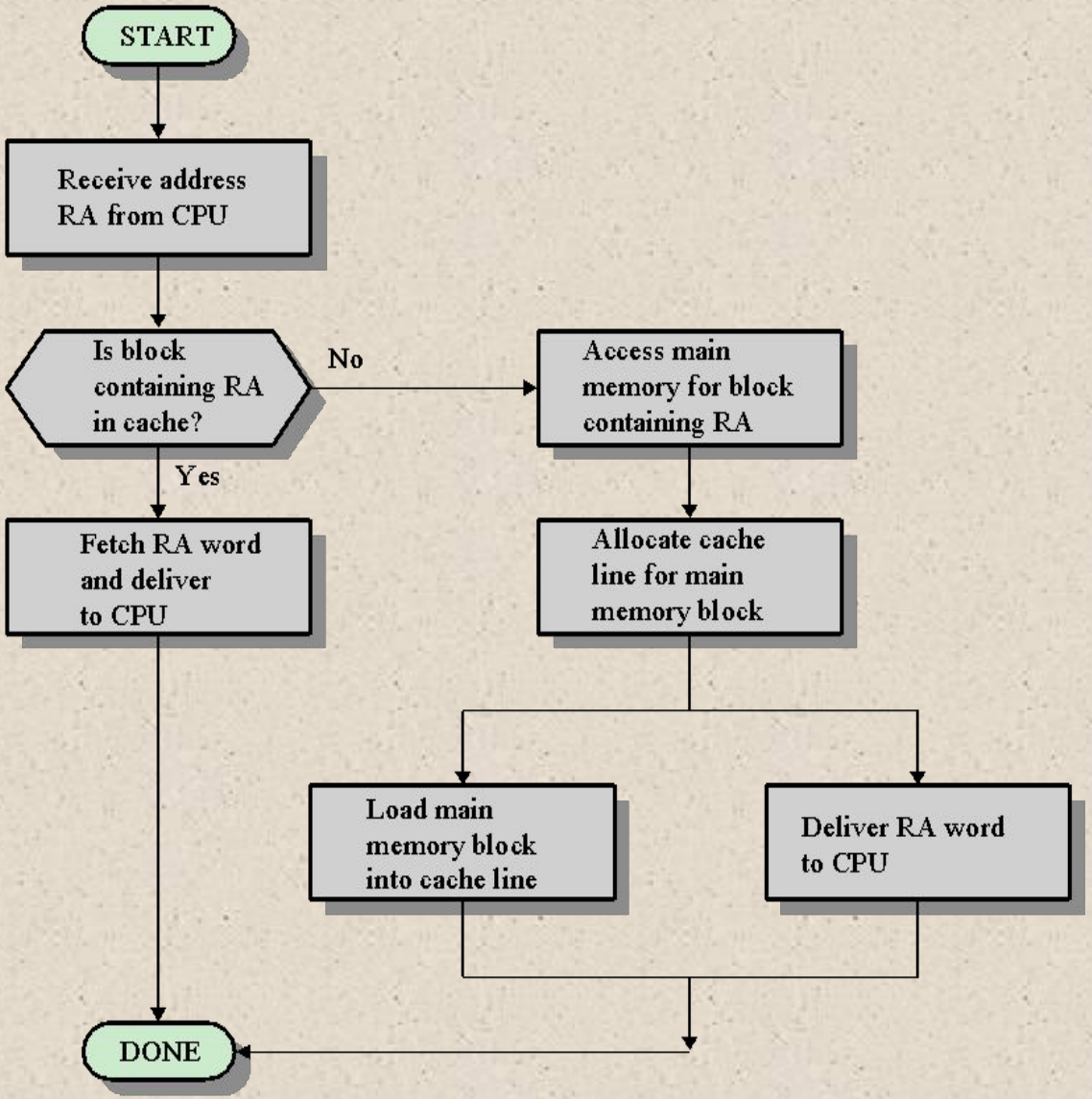


Figure 4.5 Cache Read Operation

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

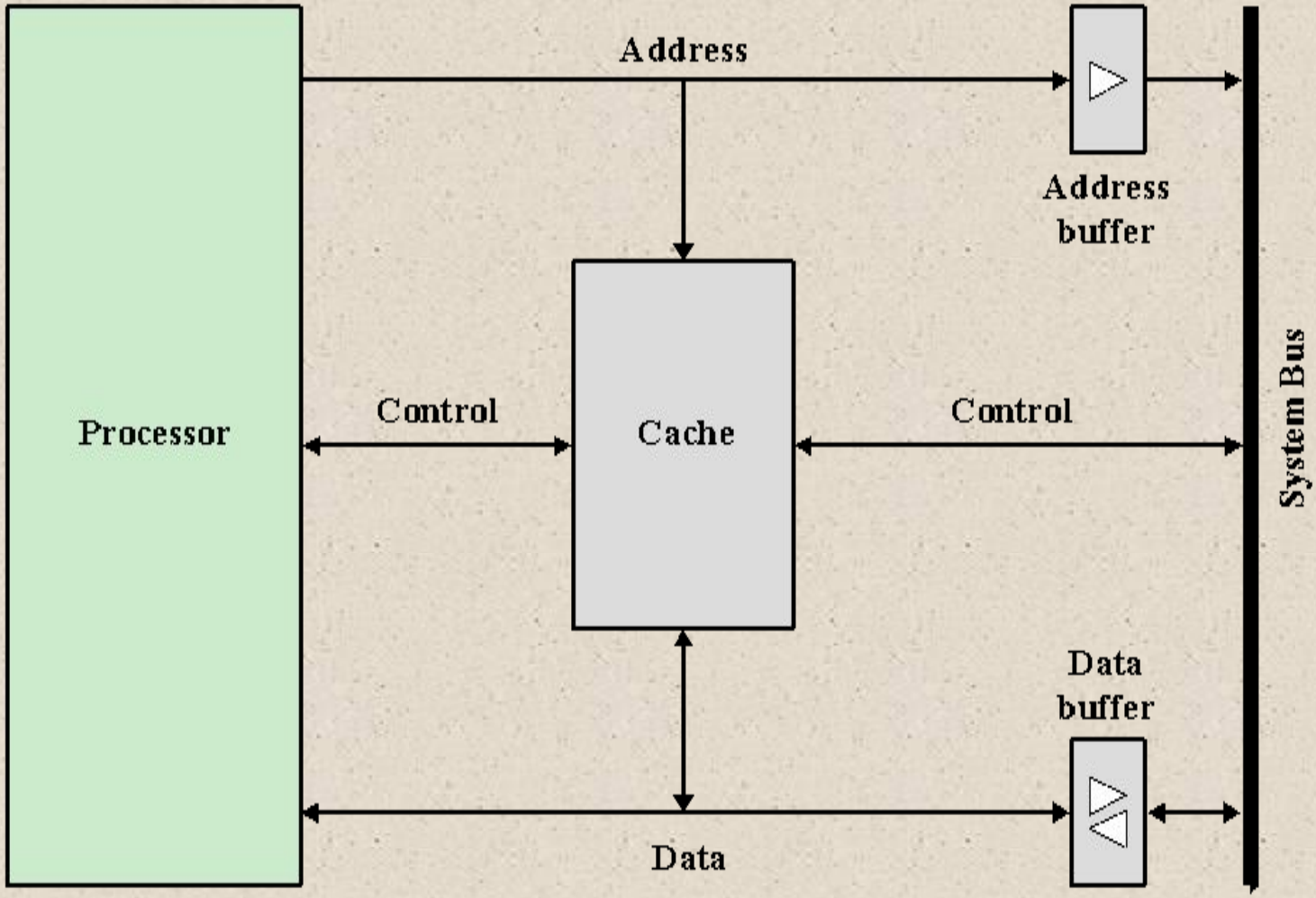


Figure 4.6 Typical Cache Organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Cache Addresses	**Write Policy**
Logical	Write through
Physical	Write back
Cache Size	**Line Size**
Mapping Function	**Number of caches**
Direct	Single or two level
Associative	Unified or split
Set Associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

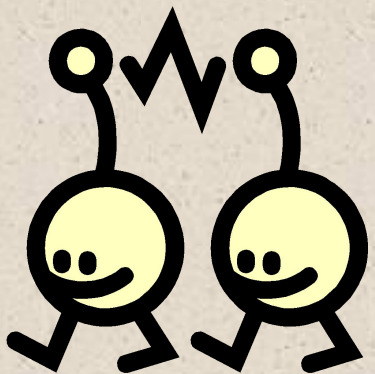
Table 4.2
Elements of Cache Design

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

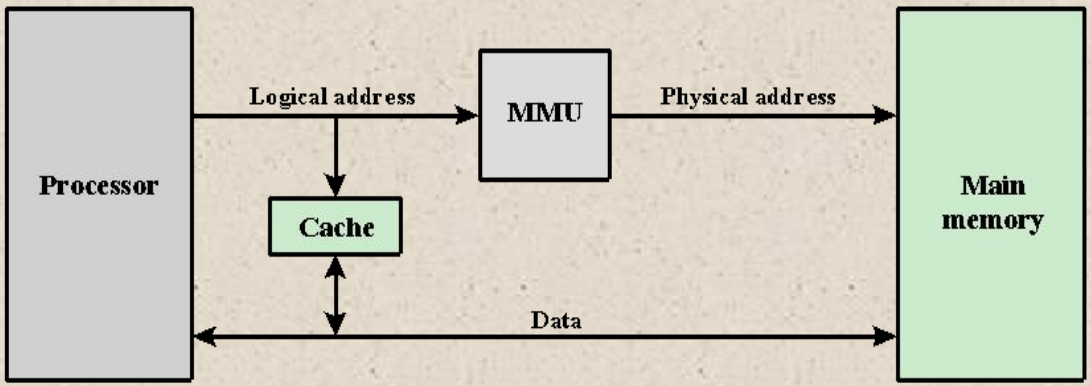
Cache Addresses

Virtual Memory

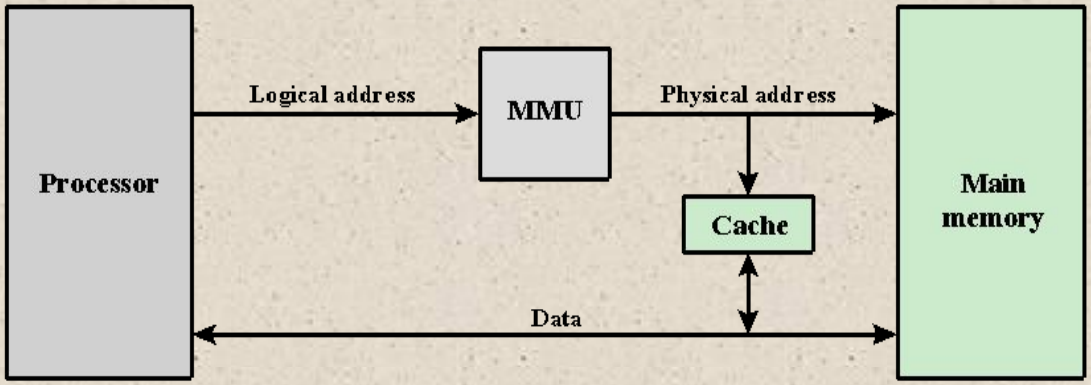
- Virtual memory
  - Facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available
  - When used, the address fields of machine instructions contain virtual addresses
  - For reads to and writes from main memory, a hardware memory management unit (MMU) translates each virtual address into a physical address in main memory



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.



(a) Logical Cache



(b) Physical Cache

Figure 4.7 Logical and Physical Caches

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Processor	Type	Year of Introduction	L1 Cachea	L2 cache	L3 Cache
IBM 360/85	Mainframe	1968	16 to 32 kB	—	—
PDP-11/70	Minicomputer	1975	1 kB	—	—
VAX 11/780	Minicomputer	1978	16 kB	—	—
IBM 3033	Mainframe	1978	64 kB	—	—
IBM 3090	Mainframe	1985	128 to 256 kB	—	—
Intel 80486	PC	1989	8 kB	—	—
Pentium	PC	1993	8 kB/8 kB	256 to 512 KB	—
PowerPC 601	PC	1993	32 kB	—	—
PowerPC 620	PC	1996	32 kB/32 kB	—	—
PowerPC G4	PC/server	1999	32 kB/32 kB	256 KB to 1 MB	2 MB
IBM S/390 G6	Mainframe	1999	256 kB	8 MB	—
Pentium 4	PC/server	2000	8 kB/8 kB	256 KB	—
IBM SP	High-end server/ supercomputer	2000	64 kB/32 kB	8 MB	—
CRAY MTAb	Supercomputer	2000	8 kB	2 MB	—
Itanium	PC/server	2001	16 kB/16 kB	96 KB	4 MB
Itanium 2	PC/server	2002	32 kB	256 KB	6 MB
IBM POWER5	High-end server	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	Supercomputer	2004	64 kB/64 kB	1MB	—
IBM POWER6	PC/server	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24-48 MB
Intel Core i7 EE 990	Workstaton/ server	2011	6 × 32 kB/32 kB	1.5 MB	12 MB
IBM zEnterprise 196	Mainframe/ Server	2011	24 × 64 kB/ 128 kB	24 × 1.5 MB	24 MB L3 192 MB L4

Table 4.3

Cache Sizes of Some Processors

[a] Two values separated by a slash refer to instruction and data caches.

[b] Both caches are instruction only; no data caches.

(Table can be found on page 134 in the textbook.)

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Mapping Function

- Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines
- Three techniques can be used:

Direct

- The simplest technique
- Maps each block of main memory into only one possible cache line

Associative

- Permits each main memory block to be loaded into any line of the cache
- The cache control logic interprets a memory address simply as a Tag and a Word field
- To determine whether a block is in the cache, the cache control logic must simultaneously examine every line's Tag for a match

Set Associative

- A compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

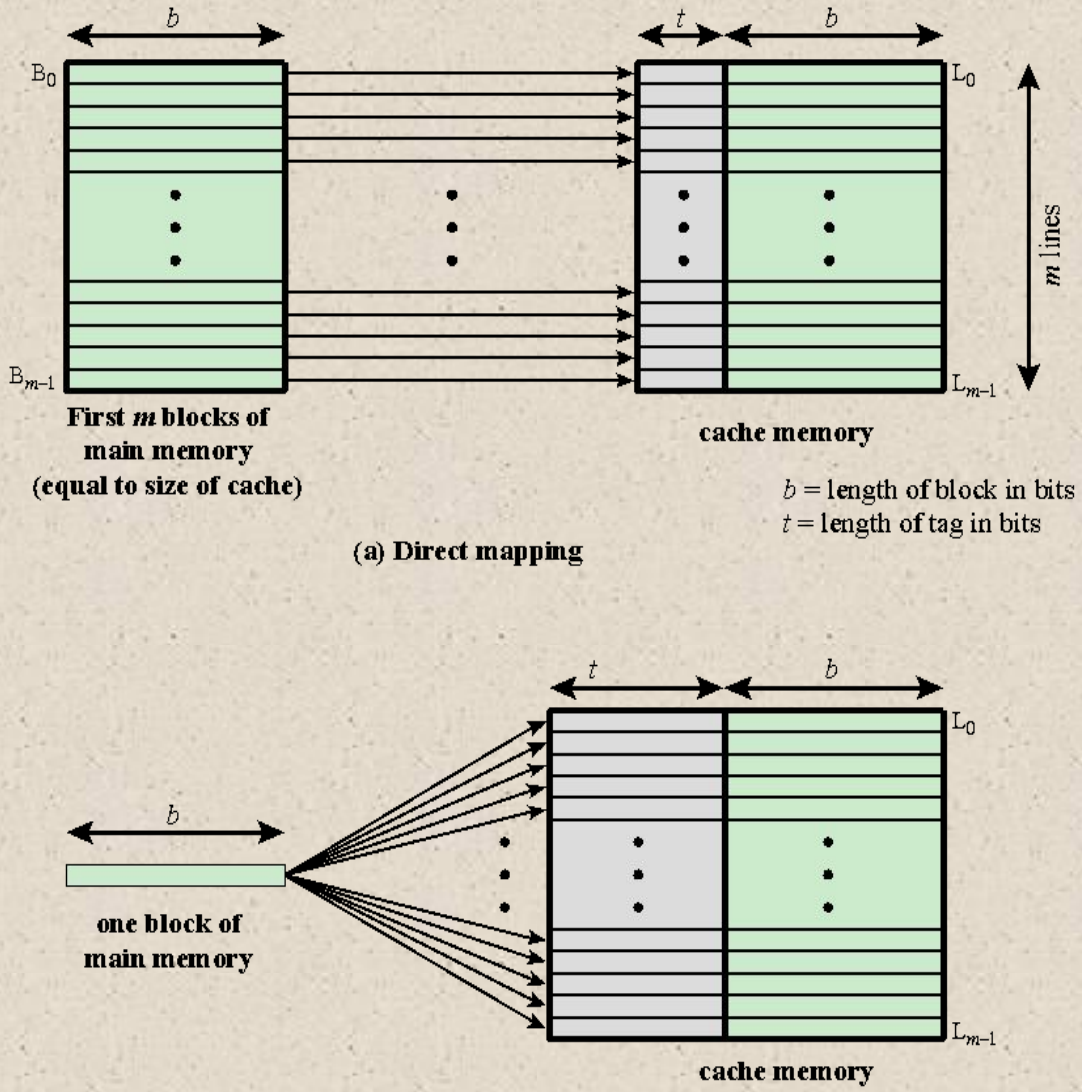


Figure 4.8 Mapping From Main Memory to Cache: Direct and Associative

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

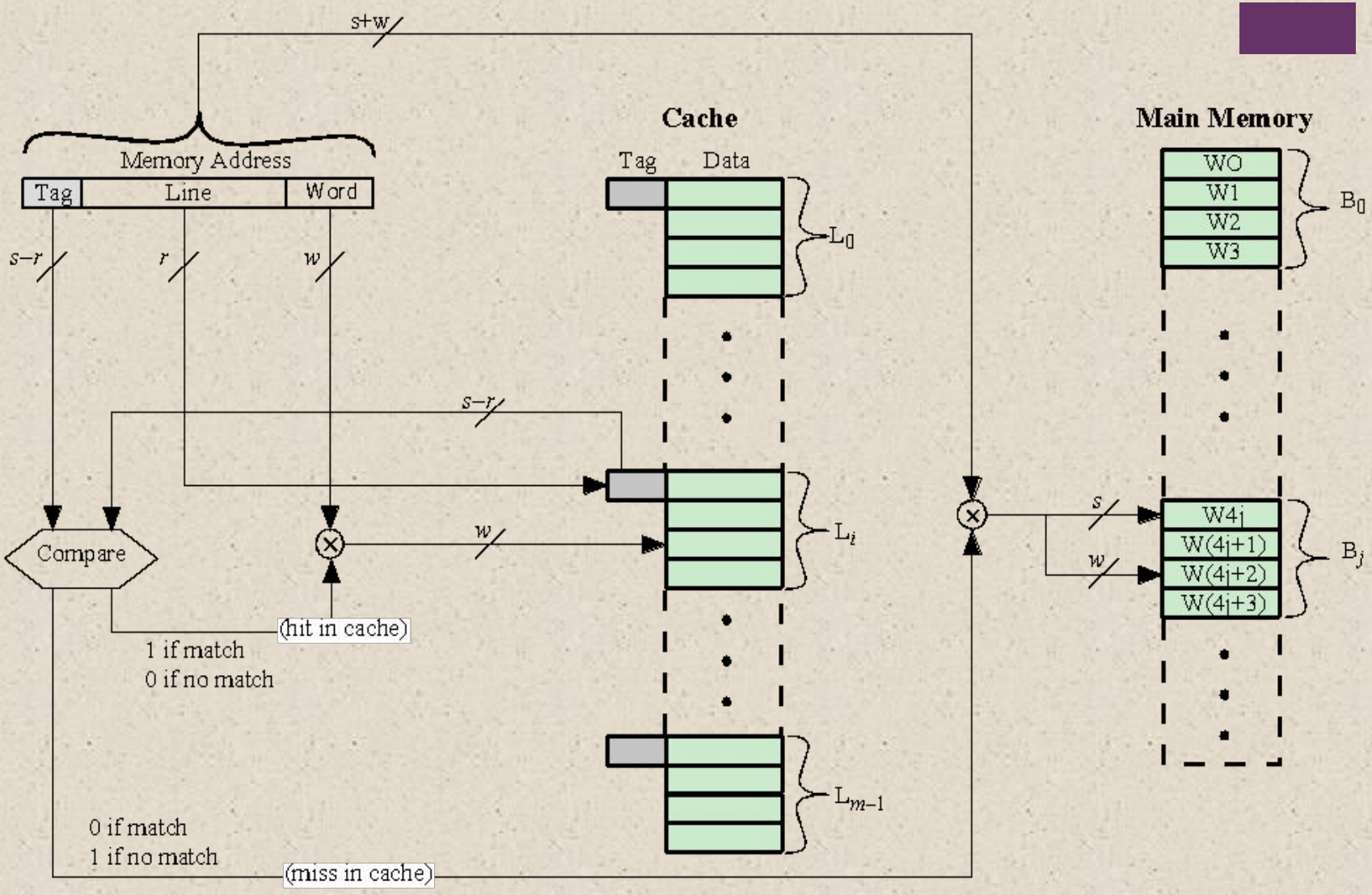


Figure 4.9 Direct-Mapping Cache Organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

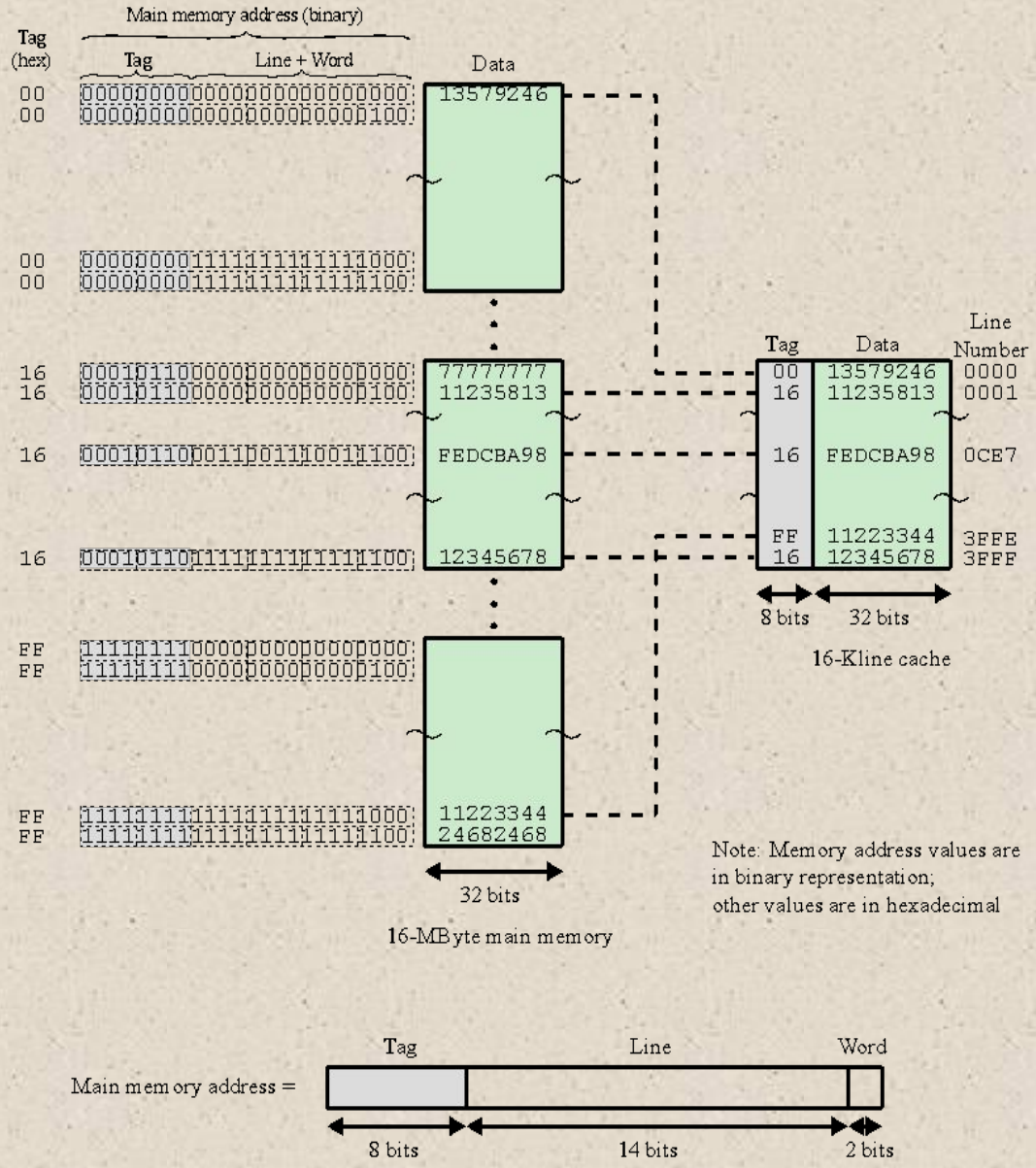


Figure 4.10 Direct Mapping Example

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Direct Mapping Summary

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = $2^{s+w}/2^{w} = 2^{s}$
- Number of lines in cache = m = 2^{r}
- Size of tag = (s – r) bits



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Victim Cache



- Originally proposed as an approach to reduce the conflict misses of direct mapped caches without affecting its fast access time
- Fully associative cache
- Typical size is 4 to 16 cache lines
- Residing between direct mapped L1 cache and the next level of memory

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

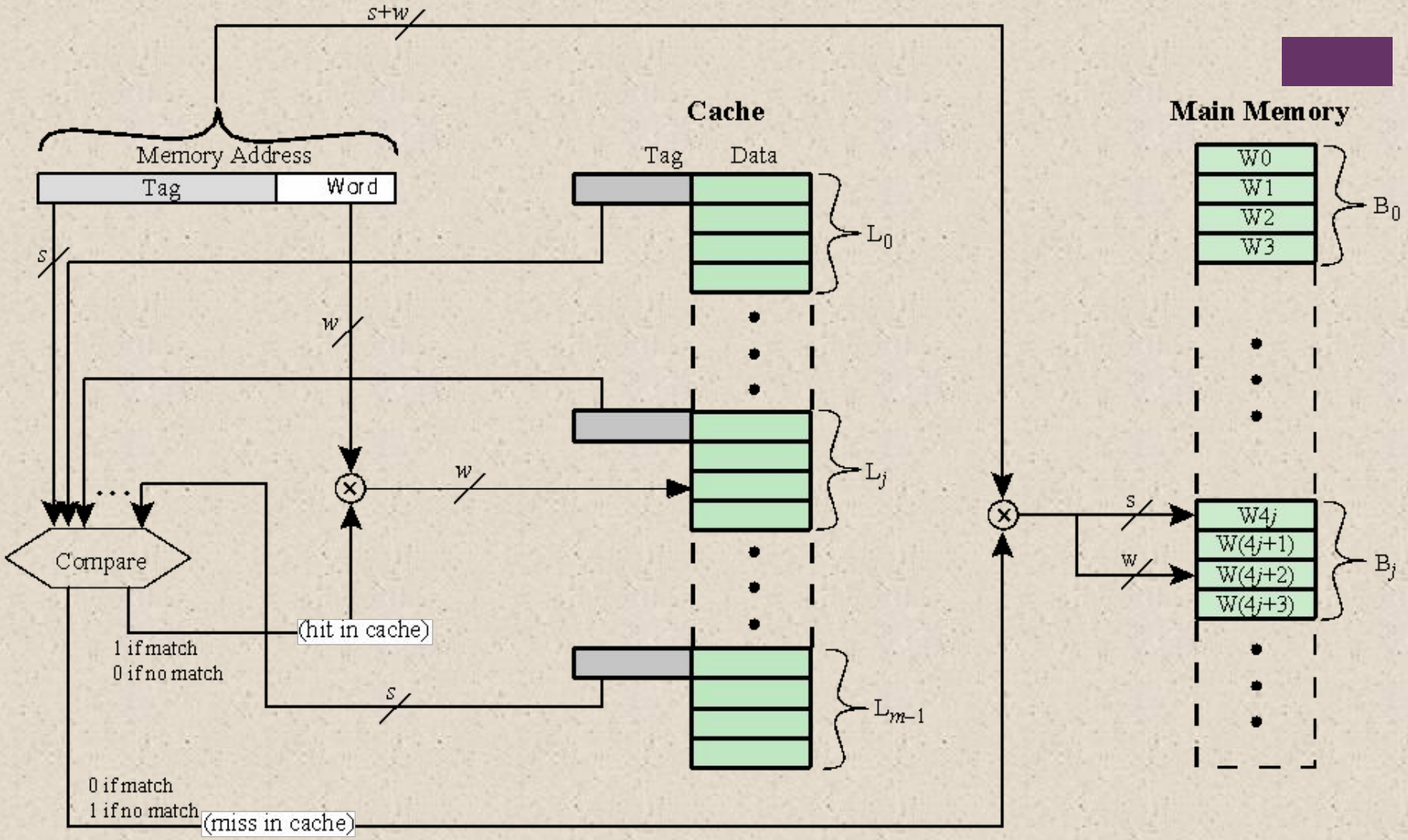


Figure 4.11 Fully Associative Cache Organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

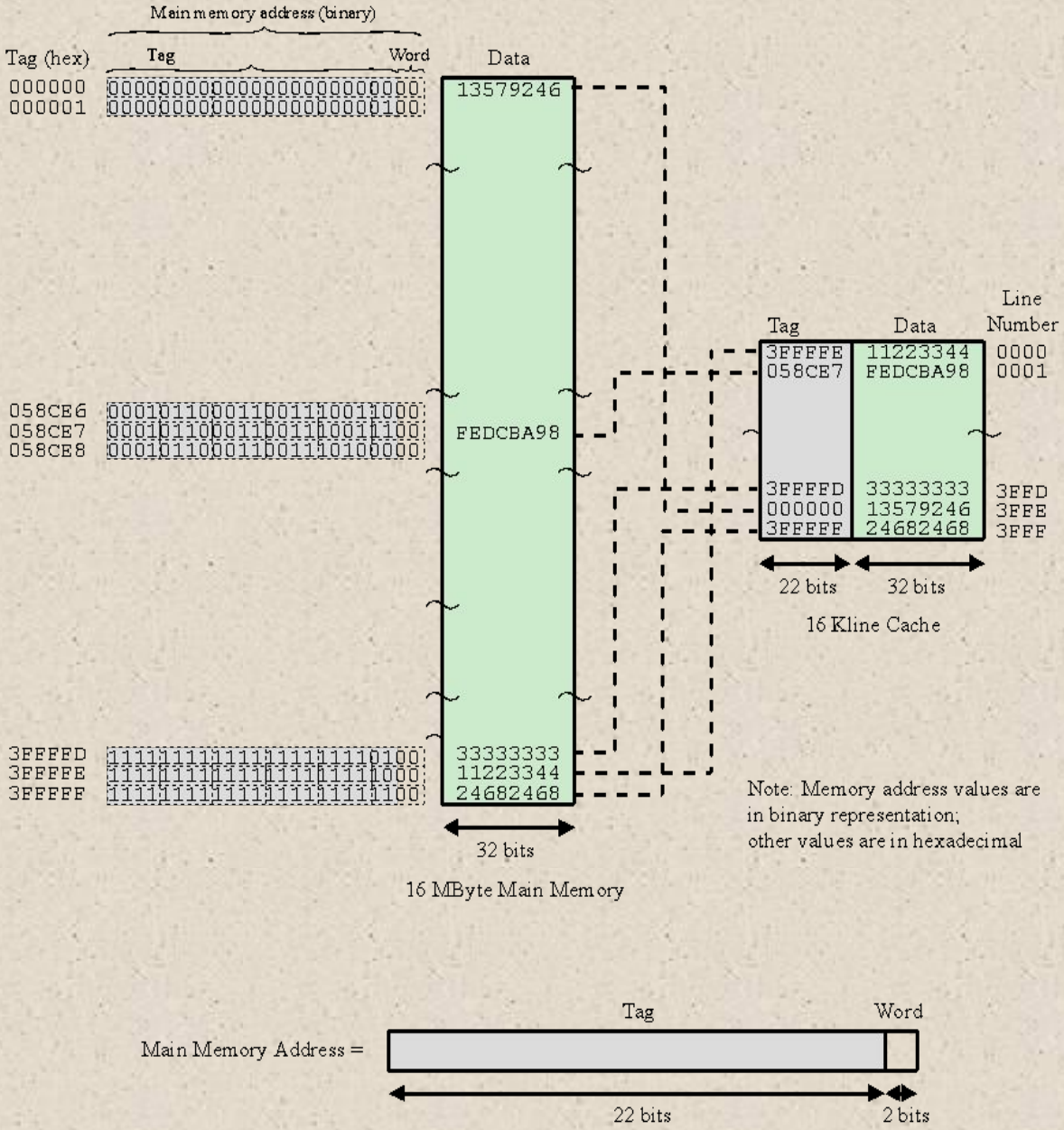


Figure 4.12 Associative Mapping Example

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Associative Mapping Summary

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = $2^{s+w}/2^{w} = 2^{s}$
- Number of lines in cache = undetermined
- Size of tag = s bits

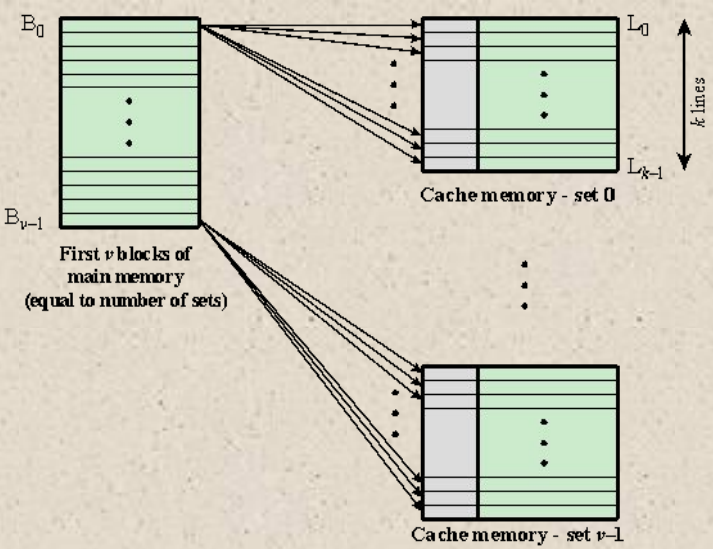


© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

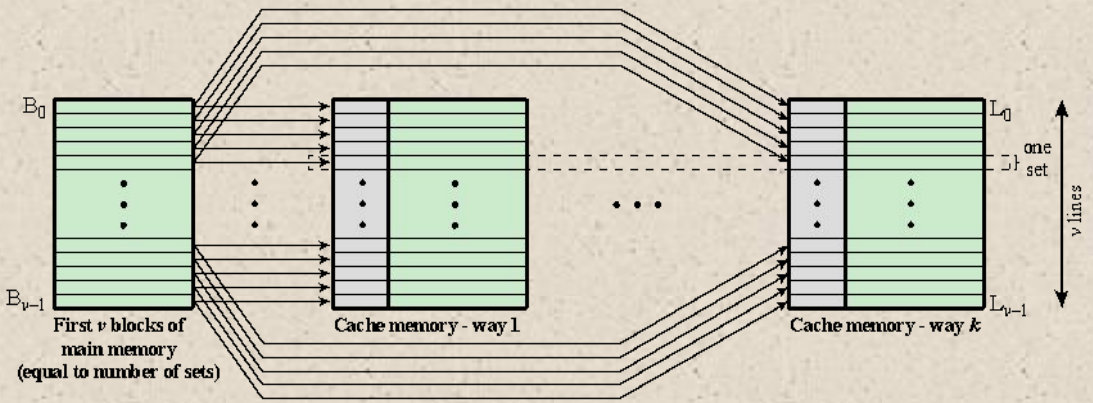
Set Associative Mapping

- Compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages
- Cache consists of a number of sets
- Each set contains a number of lines
- A given block maps to any line in a given set
- e.g. 2 lines per set
  - 2 way associative mapping
  - A given block can be in one of 2 lines in only one set

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.



(a) v associative-mapped caches



(b) k direct-mapped caches

Figure 4.13 Mapping From Main Memory to Cache: *k*-way Set Associative

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

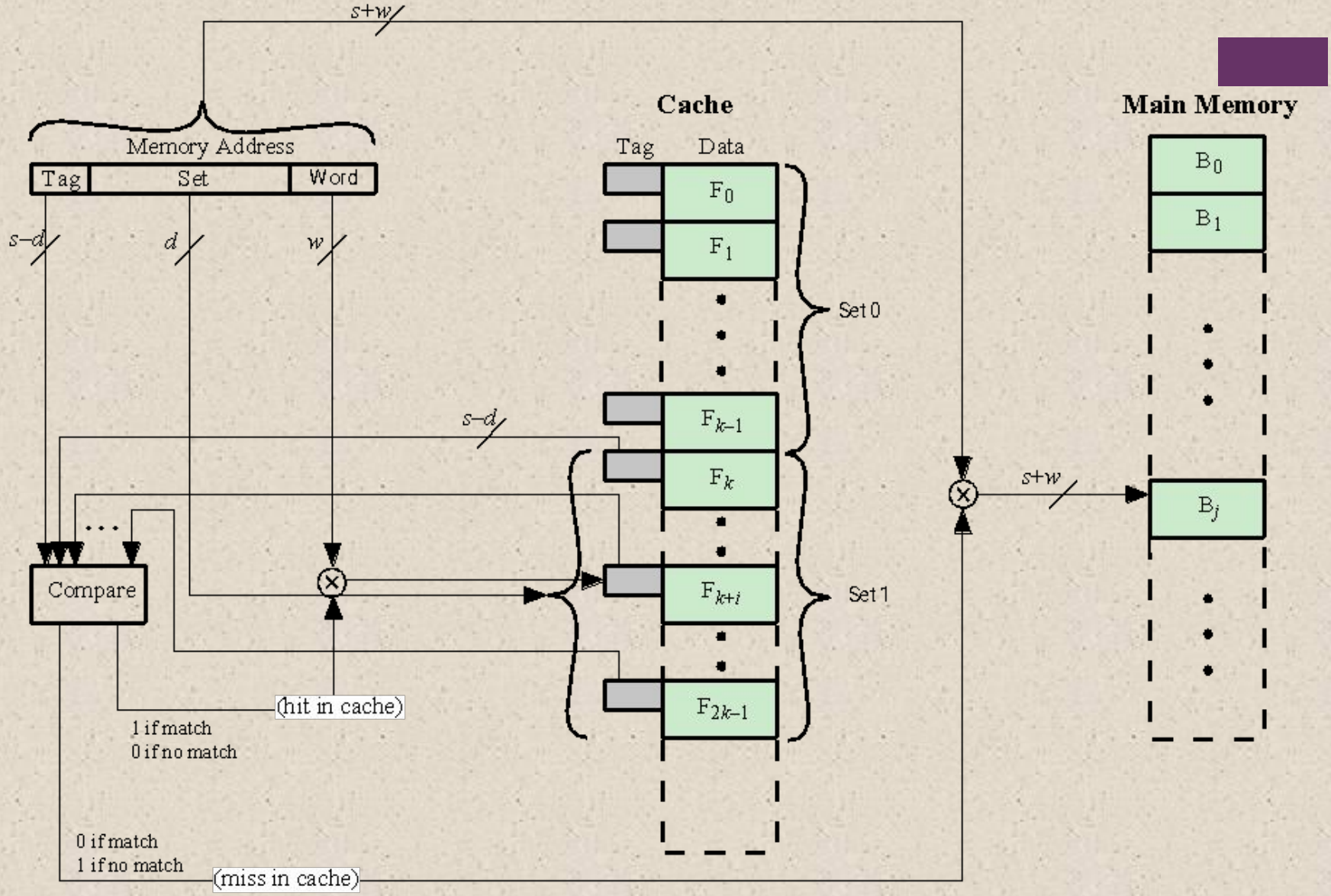


Figure 4.14 *k*-Way Set Associative Cache Organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Set Associative Mapping Summary

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = $2^{s+w}/2^{w}=2^{s}$
- Number of lines in set = k
- Number of sets = v = 2^{d}
- Number of lines in cache = m=kv = k * 2^{d}
- Size of cache = $k * 2^{d+w}$ words or bytes
- Size of tag = (s – d) bits



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

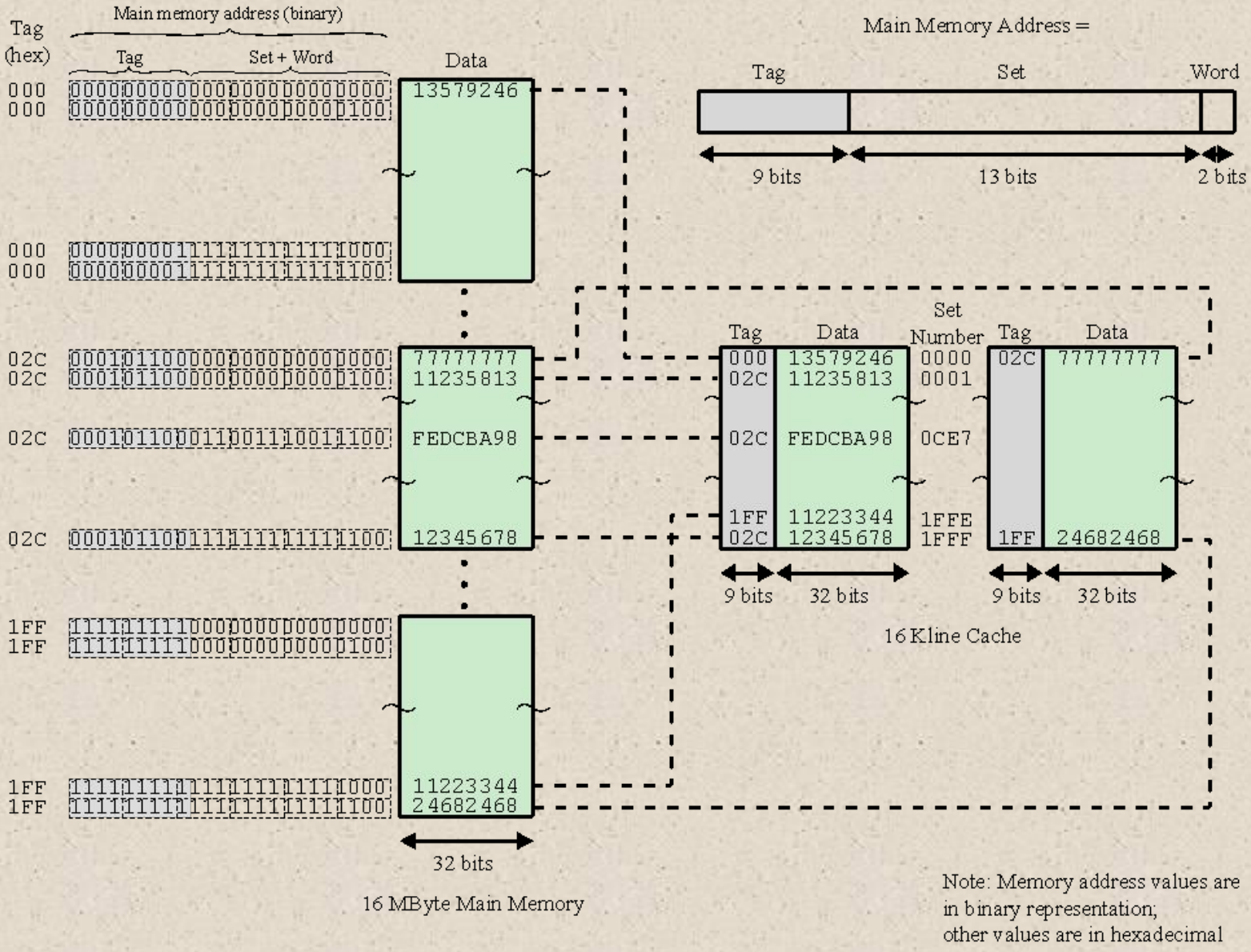


Figure 4.15 Two-Way Set Associative Mapping Example

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

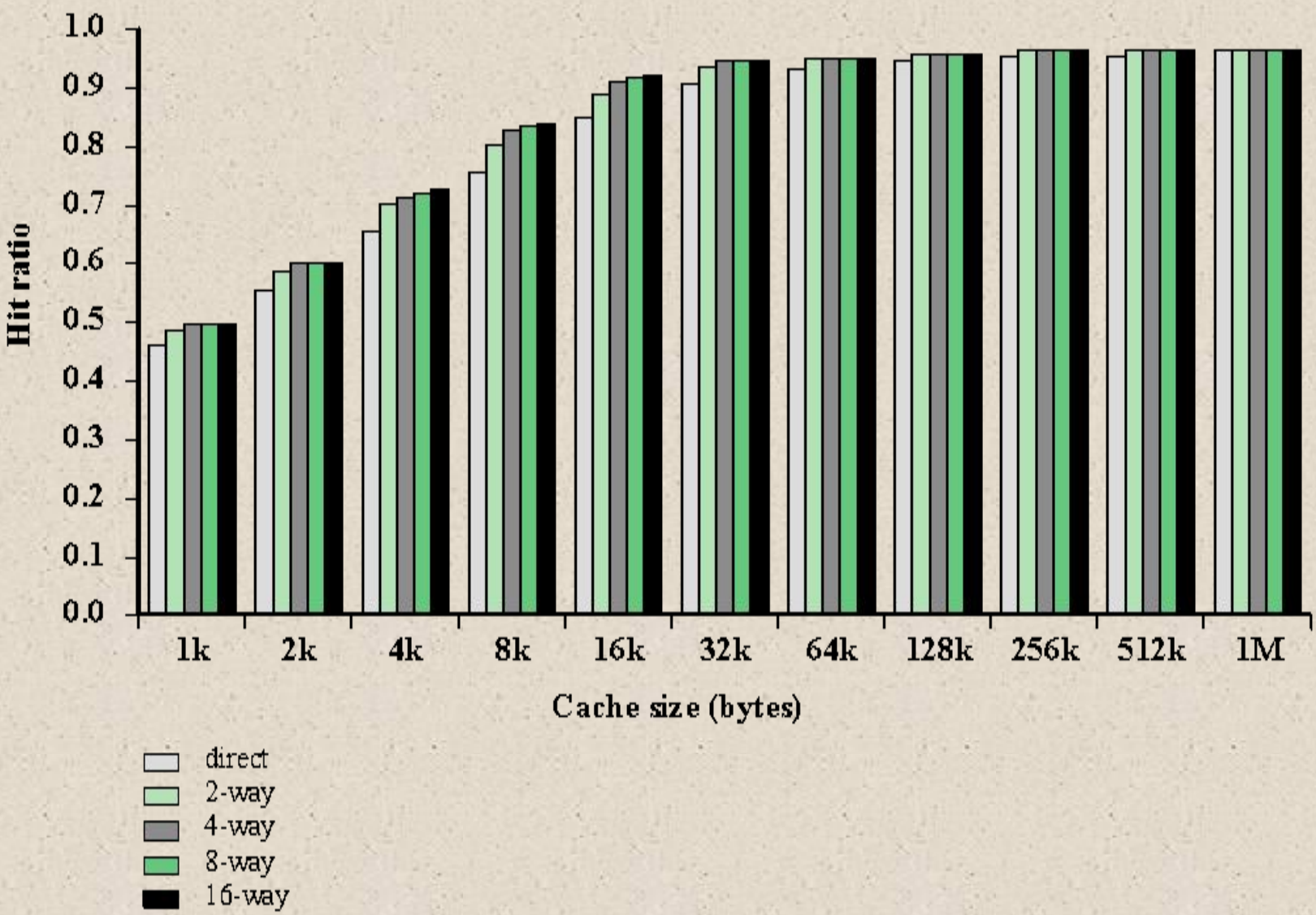
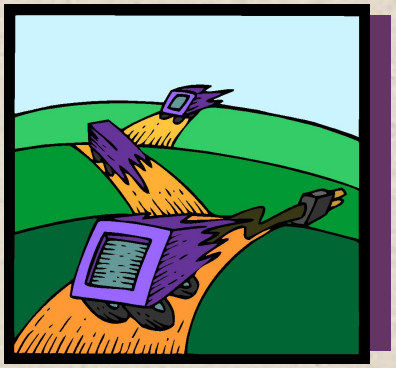


Figure 4.16 Varying Associativity over Cache Size

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Replacement Algorithms



- Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced
- For direct mapping there is only one possible line for any particular block and no choice is possible
- For the associative and set-associative techniques a replacement algorithm is needed
- To achieve high speed, an algorithm must be implemented in hardware

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

The most common replacement algorithms are:

- Least recently used (LRU)
  - Most effective
  - Replace that block in the set that has been in the cache longest with no reference to it
  - Because of its simplicity of implementation, LRU is the most popular replacement algorithm
- First-in-first-out (FIFO)
  - Replace that block in the set that has been in the cache longest
  - Easily implemented as a round-robin or circular buffer technique
- Least frequently used (LFU)
  - Replace that block in the set that has experienced the fewest references
  - Could be implemented by associating a counter with each line

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Write Policy

When a block that is resident in the cache is to be replaced there are two cases to consider:

If the old block in the cache has not been altered then it may be overwritten with a new block without first writing out the old block

If at least one write operation has been performed on a word in that line of the cache then main memory must be updated by writing the line of cache out to the block of memory before bringing in the new block

There are two problems to contend with:

More than one device may have access to main memory

A more complex problem occurs when multiple processors are attached to the same bus and each processor has its own local cache - if a word is altered in one cache it could conceivably invalidate a word in other caches

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Write Through and Write Back

- Write through
  - Simplest technique
  - All write operations are made to main memory as well as to the cache
  - The main disadvantage of this technique is that it generates substantial memory traffic and may create a bottleneck
- Write back
  - Minimizes memory writes
  - Updates are made only in the cache
  - Portions of main memory are invalid and hence accesses by I/O modules can be allowed only through the cache
  - This makes for complex circuitry and a potential bottleneck

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Line Size

When a block of data is retrieved and placed in the cache not only the desired word but also some number of adjacent words are retrieved

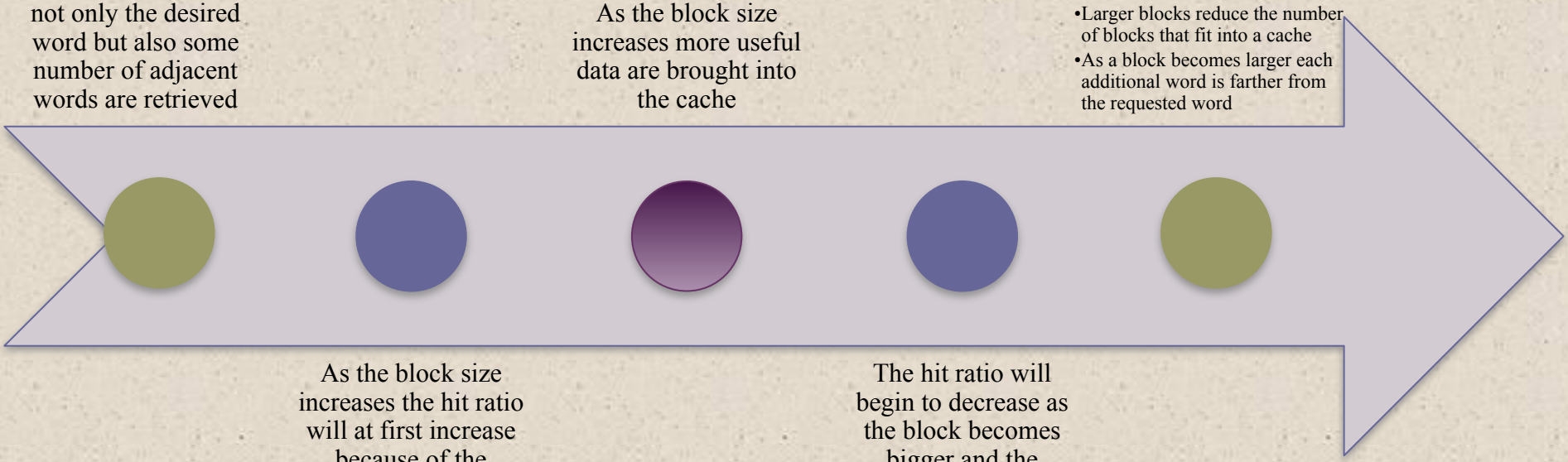
As the block size increases the hit ratio will at first increase because of the principle of locality

As the block size increases more useful data are brought into the cache

The hit ratio will begin to decrease as the block becomes bigger and the probability of using the newly fetched information becomes less than the probability of reusing the information that has to be replaced

Two specific effects come into play:

- Larger blocks reduce the number of blocks that fit into a cache
- As a block becomes larger each additional word is farther from the requested word



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Multilevel Caches

- As logic density has increased it has become possible to have a cache on the same chip as the processor
- The on-chip cache reduces the processor's external bus activity and speeds up execution time and increases overall system performance
  - When the requested instruction or data is found in the on-chip cache, the bus access is eliminated
  - On-chip cache accesses will complete appreciably faster than would even zero-wait state bus cycles
  - During this period the bus is free to support other transfers
- Two-level cache:
  - Internal cache designated as level 1 (L1)
  - External cache designated as level 2 (L2)
- Potential savings due to the use of an L2 cache depends on the hit rates in both the L1 and L2 caches
- The use of multilevel caches complicates all of the design issues related to caches, including size, replacement algorithm, and write policy

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

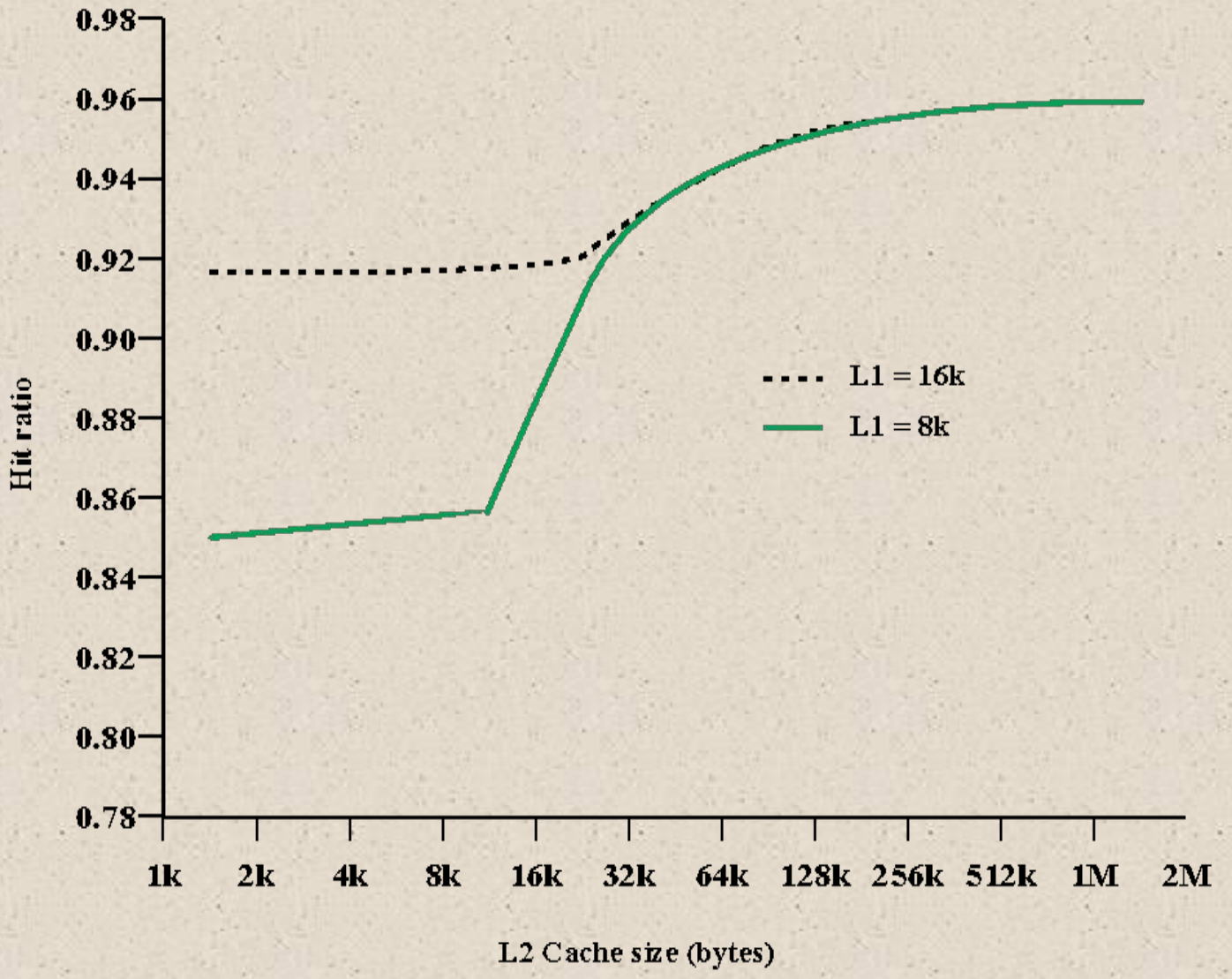


Figure 4.17 Total Hit Ratio (L1 and L2) for 8 Kbyte and 16 Kbyte L1

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Unified Versus Split Caches

- Has become common to split cache:
  - One dedicated to instructions
  - One dedicated to data
  - Both exist at the same level, typically as two L1 caches
- Advantages of unified cache:
  - Higher hit rate
    - Balances load of instruction and data fetches automatically
    - Only one cache needs to be designed and implemented
- Trend is toward split caches at the L1 and unified caches for higher levels
- Advantages of split cache:
  - Eliminates cache contention between instruction fetch/decode unit and execution unit
    - Important in pipelining

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Problem	Solution	Processor on which Feature First Appears
External memory slower than the system bus.	Add external cache using faster memory technology.	386
Increased processor speed results in external bus becoming a bottleneck for cache access.	Move external cache on-chip, operating at the same speed as the processor.	486
Internal cache is rather small, due to limited space on chip	Add external L2 cache using faster technology than main memory	486
Contention occurs when both the Instruction Prefetcher and the Execution Unit simultaneously require access to the cache. In that case, the Prefetcher is stalled while the Execution Unit's data access takes place.	Create separate data and instruction caches.	Pentium
Increased processor speed results in external bus becoming a bottleneck for L2 cache access.	Create separate back-side bus that runs at higher speed than the main (front-side) external bus. The BSB is dedicated to the L2 cache.	Pentium Pro
	Move L2 cache on to the processor chip.	Pentium II
Some applications deal with massive databases and must have rapid access to large amounts of data. The on-chip caches are too small.	Add external L3 cache.	Pentium III
	Move L3 cache on-chip.	Pentium 4

Table 4.4

Intel Cache Evolution

(Table is on page 150 in the textbook.)

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

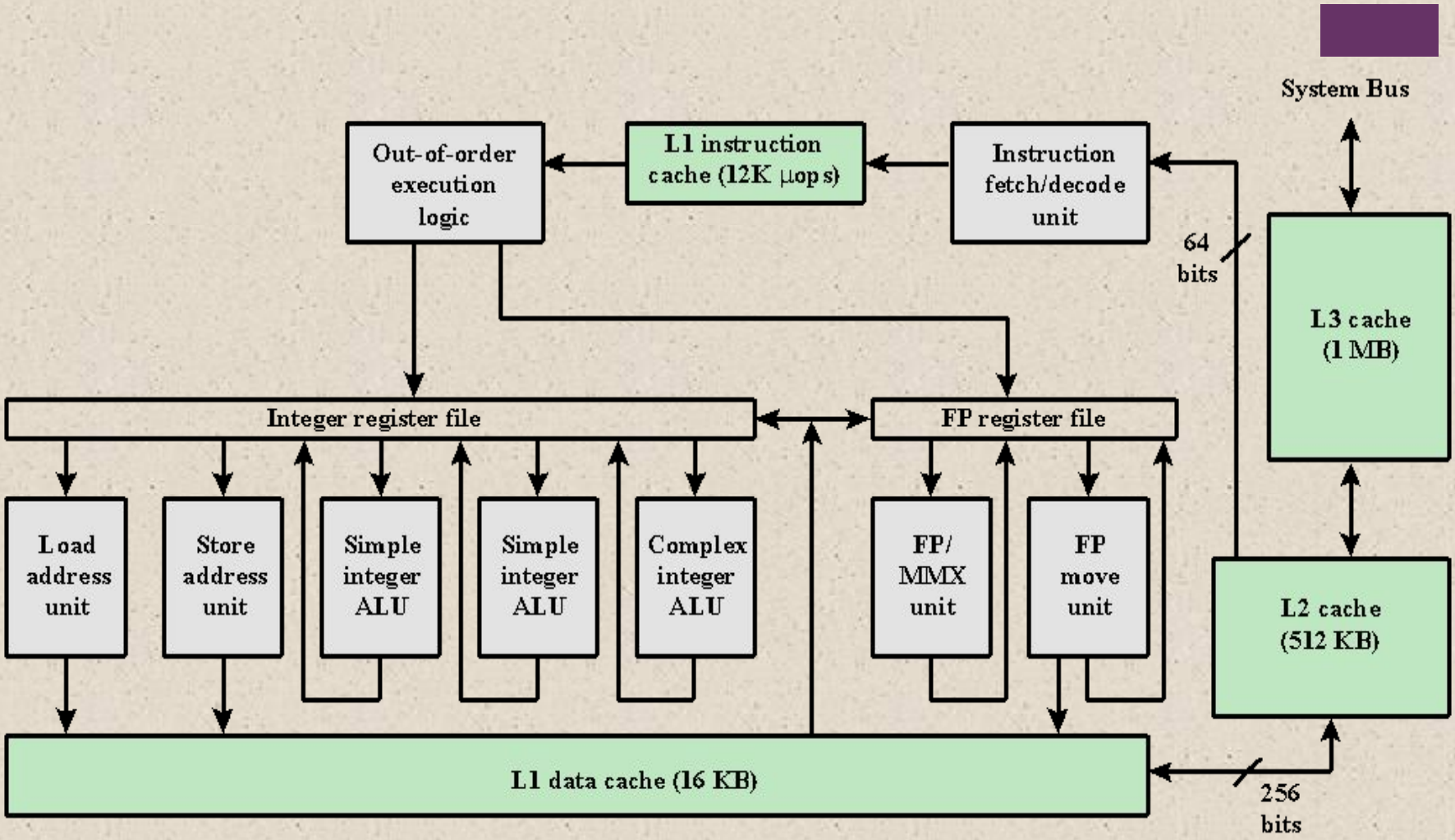


Figure 4.18 Pentium 4 Block Diagram

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Table 4.5 Pentium 4 Cache Operating Modes

Control Bits		Operating Mode		
CD	NW	Cache Fills	Write Throughs	Invalidates
0	0	Enabled	Enabled	Enabled
1	0	Disabled	Enabled	Enabled
1	1	Disabled	Disabled	Disabled

Note: CD = 0; NW = 1 is an invalid combination.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Summary

Chapter 4

- Computer memory system overview
  - Characteristics of Memory Systems
  - Memory Hierarchy
- Cache memory principles
- Pentium 4 cache organization

Cache Memory

- Elements of cache design
  - Cache addresses
  - Cache size
  - Mapping function
  - Replacement algorithms
  - Write policy
  - Line size
  - Number of caches

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.